

Data Cleaning & Preparation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load datasets
claims = pd.read_excel("claims.xlsx")
policyholders = pd.read_excel("policyholders.xlsx")
third_parties = pd.read_excel("third_parties.xlsx")
```

```
# Make a copy of all the three datasets to avoid modifying the original/Preserve raw datasets
claims_raw = claims.copy()
policyholders_raw = policyholders.copy()
third_parties_raw = third_parties.copy()
```

```
claims.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Claim_ID              8000 non-null  object
1   Policy_ID             8000 non-null  object
2   Accident_Date         8000 non-null  datetime64[ns]
3   FNOL_Date             8000 non-null  datetime64[ns]
4   Claim_Type            8000 non-null  object
5   Claim_Complexity      8000 non-null  object
6   Fraud_Flag            8000 non-null  bool
7   Litigation_Flag       8000 non-null  bool
8   Estimated_Claim_Amount 8000 non-null  int64
9   Ultimate_Claim_Amount 7575 non-null  float64
10  Severity_Band         8000 non-null  object
11  Settlement_Date       7575 non-null  datetime64[ns]
12  Status                8000 non-null  object
dtypes: bool(2), datetime64[ns](3), float64(1), int64(1), object(6)
memory usage: 703.3+ KB
```

```
policyholders.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Policy_ID             5000 non-null  object
1   Customer_ID           5000 non-null  object
2   Age_of_Driver         5000 non-null  int64
3   Gender                5000 non-null  object
4   Occupation            5000 non-null  object
5   Region                5000 non-null  object
6   Annual_Mileage        5000 non-null  int64
7   Driving_Experience_Years 5000 non-null  int64
8   Vehicle_Type          5000 non-null  object
9   Vehicle_Age          5000 non-null  int64
10  Credit_Score_Band     5000 non-null  object
dtypes: int64(4), object(7)
memory usage: 429.8+ KB
```

```
third_parties.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2410 entries, 0 to 2409
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Claim_ID              2410 non-null   object
1   TP_ID                 2410 non-null   object
2   ThirdParty_Role       2410 non-null   object
3   TP_Injury_Severity    2410 non-null   object
dtypes: object(4)
memory usage: 75.4+ KB
```

```
# Data types-Change Data type(Object to Category) Optimising Memory Usage & Performance

#Claims
claims["Claim_Type"]=claims["Claim_Type"].astype("category")
claims["Claim_Complexity"]=claims["Claim_Complexity"].astype("category")
claims["Severity_Band"]=claims["Severity_Band"].astype("category")
claims["Status"]=claims["Status"].astype("category")

# Policyholders
policyholders["Gender"]=policyholders["Gender"].astype("category")
policyholders["Occupation"]=policyholders["Occupation"].astype("category")
policyholders["Region"]=policyholders["Region"].astype("category")
policyholders["Vehicle_Type"]=policyholders["Vehicle_Type"].astype("category")
policyholders["Credit_Score_Band"]=policyholders["Credit_Score_Band"].astype("category")

# Third parties
third_parties["ThirdParty_Role"]=third_parties["ThirdParty_Role"].astype("category")
third_parties["TP_Injury_Severity"]=third_parties["TP_Injury_Severity"].astype("category")
```

Calculate Days to FNOL

Days_To_FNOL represents the reporting delay between the accident date and the first notification of loss. This feature captures behavioural and operational aspects of claims reporting and is a known indicator of claim complexity and cost.

```
# Create a column to calculate the interval between FNOL Date and Accident Date

claims["Days_To_FNOL"] = (
    claims["FNOL_Date"] - claims["Accident_Date"]
).dt.days
```

```
# Count the no of non-null values for data integrity or accuracy

claims["Days_To_FNOL"].isna().value_counts()
```

```
Days_To_FNOL
False      8000
Name: count, dtype: int64
```

Standardize categorical values

```
# Standardize ordinal features
severity_map = {"Minor": 1, "Serious": 2, "Fatal": 3}

# Clean and map TP_Injury_Severity
third_parties["TP_Injury_Severity"] = third_parties["TP_Injury_Severity"].str.strip().str.title()
third_parties["Injury_Severity_Code"] = third_parties["TP_Injury_Severity"].map(severity_map)
```

Standardize categorical values for all datasets

```
# Function to clean string categories
def clean_categories(df, cols):
    for col in cols:
        df[col] = df[col].astype(str).str.strip().str.title()
    return df

# Claims dataset
claims_nominal_cols = ["Claim_Type", "Claim_Complexity", "Severity_Band", "Status"]
claims = clean_categories(claims, claims_nominal_cols)

# Policyholders dataset
policyholders_nominal_cols = ["Gender", "Occupation", "Region", "Vehicle_Type", "Credit_Score_Band"]
policyholders = clean_categories(policyholders, policyholders_nominal_cols)

# Third parties dataset
third_parties_nominal_cols = ["ThirdParty_Role"]
third_parties = clean_categories(third_parties, third_parties_nominal_cols)
```

```
# Check the results
print("Claims categorical value counts:")
for col in claims_nominal_cols:
    print(f"\n{col}:\n", claims[col].value_counts())

print("\nPolicyholders categorical value counts:")
for col in policyholders_nominal_cols:
    print(f"\n{col}:\n", policyholders[col].value_counts())

print("\nThird parties categorical value counts:")
for col in third_parties_nominal_cols + ["TP_Injury_Severity", "Injury_Severity_Code"]:
    print(f"\n{col}:\n", third_parties[col].value_counts())
```

Claims categorical value counts:

```
Claim_Type:
Claim_Type
Collision    4394
Weather      1193
Theft         811
Vandalism     773
Fire          425
Other         404
Name: count, dtype: int64
```

```
Claim_Complexity:
Claim_Complexity
Low          5588
Medium       1781
High          631
Name: count, dtype: int64
```

```
Severity_Band:
Severity_Band
Minor        3989
Moderate     1992
Major        1241
Severe        619
Catastrophic  159
Name: count, dtype: int64
```

```
Status:
Status
Settled      7575
Open         425
Name: count, dtype: int64
```

Policyholders categorical value counts:

```
Gender:
Gender
Male        2522
Female      2478
Name: count, dtype: int64
```

```
Occupation:
  Occupation
Employed      1946
Retired       1288
Self-Employed 904
Unemployed    457
Student       405
Name: count, dtype: int64
```

```
Region:
  Region
Glasgow      537
Bristol       529
Liverpool     524
Edinburgh    513
London        499
Leeds         498
Cardiff       482
Newcastle     477
Manchester    477
Birmingham   464
Name: count, dtype: int64
```

```
Vehicle_Type:
  Vehicle_Type
Sedan         1541
Hatchback     1200
Suv           1046
Coupe          511
Van            458
Motorcycle     244
Name: count, dtype: int64
```

```
Credit_Score_Band:
  Credit_Score_Band
Good            1995
Excellent       1509
Fair            1006
Poor            490
Name: count, dtype: int64
```

Third parties categorical value counts:

```
ThirdParty_Role:
  ThirdParty_Role
Passenger      836
Pedestrian      797
Driver         777
Name: count, dtype: int64
```

```
TP_Injury_Severity:
  TP_Injury_Severity
Minor           2050
Serious         342
Fatal           18
Name: count, dtype: int64
```

```
Injury_Severity_Code:
  Injury_Severity_Code
1           2050
2           342
3            18
Name: count, dtype: int64
```

Remove Duplicates

```
# Check how many duplicates exist in Claims Dataset
duplicates = claims.duplicated(subset="Claim_ID").sum()
print(f"Number of duplicate Claim_IDs: {duplicates}")
```

Number of duplicate Claim_IDs: 0

```
# Count exact duplicate rows in third_parties.
# rows that are identical across all columns as one claim can involve multiple third parties, and dropping on Claim_ID would
remove valid rows.
duplicate_tp = third_parties.duplicated().sum()
print(f"Exact duplicate rows in third_parties: {duplicate_tp}")
```

Exact duplicate rows in third_parties: 0

Handle Outliers

Log Transformation is chosen over Winsorization because:

Log transformation compresses the extreme high values while stretching the low ones slightly, making the distribution closer to normal, unlike Winsorization, which caps extreme values

Log transformation keeps all claims intact. This is important in insurance because large claims carry critical financial risk information.

Log transformation is almost always better for claim cost prediction.

```
# Handle outliers-Claim costs are naturally right-skewed, so we don't delete outliers.
# Create a log-transformed column for modeling
claims["Ultimate_Claim_Amount_Log"] = np.log1p(claims["Ultimate_Claim_Amount"])
```

```
# Merge claims + policyholders
claims_merged = claims.merge(policyholders, on="Policy_ID", how="left")

# Merge with third_parties
claims_merged = claims_merged.merge(third_parties, on="Claim_ID", how="left")

# Save cleaned merged dataset
claims_merged.to_csv("cleaned_claims_dataset.csv", index=False)
```

```
# Final cleaned dataset export
claims_merged.to_csv("cleaned_claims_dataset.csv", index=False)
```

```
# Load the cleaned data
cleaned_claims = pd.read_csv("cleaned_claims_dataset.csv")

# Print info about columns, data types, and non-null counts
cleaned_claims.info()

# Print the shape of the DataFrame (rows, columns)
print("Shape of cleaned data:", cleaned_claims.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8413 entries, 0 to 8412
Data columns (total 29 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Claim_ID              8413 non-null   object
 1   Policy_ID             8413 non-null   object
 2   Accident_Date         8413 non-null   object
 3   FNOL_Date             8413 non-null   object
 4   Claim_Type            8413 non-null   object
 5   Claim_Complexity      8413 non-null   object
 6   Fraud_Flag            8413 non-null   bool
 7   Litigation_Flag       8413 non-null   bool
 8   Estimated_Claim_Amount 8413 non-null   int64
 9   Ultimate_Claim_Amount 7968 non-null   float64
10   Severity_Band         8413 non-null   object
11   Settlement_Date       7968 non-null   object
12   Status                8413 non-null   object
```

```
13 Days_To_FNOL      8413 non-null  int64
14 Ultimate_Claim_Amount_Log  7968 non-null  float64
15 Customer_ID       8413 non-null  object
16 Age_of_Driver      8413 non-null  int64
17 Gender            8413 non-null  object
18 Occupation        8413 non-null  object
19 Region            8413 non-null  object
20 Annual_Mileage     8413 non-null  int64
21 Driving_Experience_Years  8413 non-null  int64
22 Vehicle_Type       8413 non-null  object
23 Vehicle_Age       8413 non-null  int64
24 Credit_Score_Band  8413 non-null  object
25 TP_ID             2410 non-null  object
26 ThirdParty_Role    2410 non-null  object
27 TP_Injury_Severity  2410 non-null  object
28 Injury_Severity_Code  2410 non-null  float64
dtypes: bool(2), float64(3), int64(6), object(18)
memory usage: 1.7+ MB
Shape of cleaned data: (8413, 29)
```

Validation/ Observation Check

Claims ID rose from 8000 to 8413, which is as a result of one-to-many relationships- multiple third party to one claim ID

```
# Filter claims with multiple third parties
multi_tp_claims = cleaned_claims[cleaned_claims["Claim_ID"].map(cleaned_claims["Claim_ID"].value_counts()) > 1]
multi_tp_claims.sort_values("Claim_ID").head(10)

.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	Claim_ID	Policy_ID	Accident_Date	FNOL_Date	Claim_Type	Claim_Complexity	Fraud_Flag	Litigation_Flag	Estimated_Claim_Amount
16	CLM30016	POL10930	2024-05-19	2024-05-20	Collision	Low	False	True	1703
17	CLM30016	POL10930	2024-05-19	2024-05-20	Collision	Low	False	True	1703
55	CLM30054	POL11286	2020-11-14	2020-11-15	Collision	Low	False	False	3203
56	CLM30054	POL11286	2020-11-14	2020-11-15	Collision	Low	False	False	3203
85	CLM30083	POL10991	2019-05-06	2019-05-06	Collision	Low	False	False	1818
86	CLM30083	POL10991	2019-05-06	2019-05-06	Collision	Low	False	False	1818
93	CLM30090	POL14071	2021-01-30	2021-01-31	Collision	Medium	False	False	3344
94	CLM30090	POL14071	2021-01-30	2021-01-31	Collision	Medium	False	False	3344
105	CLM30101	POL11800	2025-01-04	2025-01-04	Collision	High	False	False	127927
106	CLM30101	POL11800	2025-01-04	2025-01-04	Collision	High	False	False	127927

10 rows × 29 columns

```
# No of claims with multiple third parties
cleaned_claims["Claim_ID"].value_counts().gt(1).sum()
```

413

```
# how many third parties per claim
tp_per_claim = cleaned_claims.groupby("Claim_ID")["TP_ID"].nunique()
tp_per_claim.value_counts().sort_index()
```

```
TP_ID
0    6003
1    1584
2     413
Name: count, dtype: int64
```

Some claims involve multiple third parties (e.g., driver and passengers).

To avoid duplicating claim-level outcomes, third-party information was aggregated to the claim level using summary statistics such as the number of third parties and maximum injury severity before modeling.

```
# In order not to overstate/represent claims due to multiple third parties, we aggregate third party data to claim level.
# Keeping the duplicates(multiple third parties to a single claim ID) will distort claim cost prediction
# Aggregate third-party information to claim level
tp_agg = (third_parties.groupby("Claim_ID").agg(
    Num_ThirdParties=("TP_ID", "count"),
    Max_ThirdParty_Severity=("Injury_Severity_Code", "max"),
    Any_ThirdParty_Injury=("Injury_Severity_Code", lambda x: int(x.gt(0).any()),
    Num_Passengers=("ThirdParty_Role", lambda x: (x == "Passenger").sum()),
    Num_Drivers=("ThirdParty_Role", lambda x: (x == "Driver").sum())).reset_index())
```

```
# Merge claims + policyholders
claims_final = claims.merge(policyholders, on="Policy_ID", how="left")

# Merge with aggregated third_parties
claims_final = claims_final.merge(tp_agg, on="Claim_ID", how="left")
```

```
# Check no of missing values from these columns
tp_cols = ["Num_ThirdParties", "Max_ThirdParty_Severity", "Any_ThirdParty_Injury", "Num_Passengers", "Num_Drivers"]
claims_final[tp_cols].isnull().sum()
```

```
Num_ThirdParties      6003
Max_ThirdParty_Severity 6003
Any_ThirdParty_Injury 6003
Num_Passengers        6003
Num_Drivers           6003
dtype: int64
```

```
# Check for No of missing values, that is how many claims had zero third parties
claims_final[tp_cols].isnull().any(axis=1).sum()
```

```
6003
```

```
# Fill Missing values with zero(0). which is claims with no third party
claims_final[tp_cols] = claims_final[tp_cols].fillna(0)
```

```
claims_final[tp_cols].isnull().any(axis=1).sum()
```

```
0
```

```
# Save cleaned merged(final) dataset
claims_final.to_csv("cleaned_final_claims_dataset.csv", index=False)
```

```
# Load the cleaned final data
cleaned_final_claims = pd.read_csv("cleaned_final_claims_dataset.csv")

# Print info about columns, data types, and non-null counts
cleaned_final_claims.info()

# Print the shape of the DataFrame (rows, columns)
print("Shape of cleaned_final data:", cleaned_final_claims.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Claim_ID                             8000 non-null   object
1   Policy_ID                             8000 non-null   object
2   Accident_Date                         8000 non-null   object
3   FNOL_Date                             8000 non-null   object
4   Claim_Type                             8000 non-null   object
5   Claim_Complexity                       8000 non-null   object
6   Fraud_Flag                             8000 non-null   bool
7   Litigation_Flag                       8000 non-null   bool
8   Estimated_Claim_Amount                 8000 non-null   int64
9   Ultimate_Claim_Amount                  7575 non-null   float64
10  Severity_Band                          8000 non-null   object
11  Settlement_Date                        7575 non-null   object
12  Status                                 8000 non-null   object
13  Days_To_FNOL                           8000 non-null   int64
14  Ultimate_Claim_Amount_Log              7575 non-null   float64
15  Customer_ID                           8000 non-null   object
16  Age_of_Driver                          8000 non-null   int64
17  Gender                                 8000 non-null   object
18  Occupation                             8000 non-null   object
19  Region                                 8000 non-null   object
20  Annual_Mileage                         8000 non-null   int64
21  Driving_Experience_Years               8000 non-null   int64
22  Vehicle_Type                           8000 non-null   object
23  Vehicle_Age                           8000 non-null   int64
24  Credit_Score_Band                     8000 non-null   object
25  Num_Third_Parties                      8000 non-null   float64
26  Max_ThirdParty_Severity                 8000 non-null   float64
27  Any_ThirdParty_Injury                  8000 non-null   float64
28  Num_Passengers                         8000 non-null   float64
29  Num_Drivers                            8000 non-null   float64
dtypes: bool(2), float64(7), int64(6), object(15)
memory usage: 1.7+ MB
Shape of cleaned_final data: (8000, 30)
```

